

EE 1004 & CSE 2062

OBJECT-ORIENTED PROGRAMMING

Spring 2025–2026

PROJECT REPORT

Group 6

	Name-Surname	<u>Student ID</u>	<u>Contribution Details</u>
A	Burak EFE	150723843	Code Development, GUI and Project Report
B	Furkan Karaca	150721065	Code Development, GUI and Project Report
C	Sıla Ilgıt Kılınç	150721037	Revising the Project Report, Code Development
D	Esad Özer	150724018	Revising the Project Report, Code Development
E	Baran ÇELİK	150721054	Revising the Project Report, Code Development

Table Of Contents

1-LibraryItem (Abstract Base Class) & Subclasses (Book, Magazine, DVD)	2
1.1 Brief explanation of the problem	3
1.2 UML Diagram	3
Figure 1: UML diagram of the LibraryItem inheritance hierarchy	3
1.3 Data field definition list.....	3
Table 1: Data field definition list	4
1.4 Object definition list	4
Table 2: Object definition list	4
1.5 Method definition list	4
Table 3: Method definition list	5
1.6 Class definition list.....	5
Table 4: Class definition list	5
1.7 OOP Concepts	5
1.8 Screenshots of the sequence.....	6
Figure 2: Screenshots of the sequence	6
1.9 Extra features.....	6
2- Library Class (Aggregator Class)	7
2.1 Brief explanation of the problem	7
2.2 UML Diagram	7
Figure 3: UML Diagram	7
2.3 Data field definition list.....	7
Table 5: Data field definition list	7
2.4 Object definition list	8
Table 6: Object definition list	8
2.5 Method definition list	8
Table 7: Method definition list	8
2.6 Class definition list.....	8
Table 8: Class definition list	9
2.7 OOP Concepts	9
2.8 Screenshots of the sequence.....	9
Figure 4: Screenshots of the sequence	9
2.9 Extra features.....	9
3-Main Class (Console Driver / User Interface)	10

3.1 Brief explanation of the problem	11
3.2 UML Diagram	11
Figure 5: Screenshots of the sequence	11
3.3 Data field definition list.....	11
Table 9: Data field definition list	11
3.4 Object definition list	11
Table 10: Object definition list.....	12
3.5 Method definition list	12
Table 11: Method definition list	12
3.6 Class definition list.....	12
Table 12: Class definition list	12
3.7 OOP Concepts	12
3.8 Screenshots of the sequence.....	12
Figure 6: Screenshots of the sequence	13
3.9 Extra features.....	13
4-Error Handling (Defensive Programming & Logical Guards)	13
4.1 Brief explanation of the problem	14
4.2 UML Diagram	14
Figure 7: UML Diagram	14
4.3 Data field definition list.....	14
Table 13: Data field definition list	14
4.4 Object definition list	15
Table 14: Object definition list.....	15
4.5 Method definition list	15
Table 15: Method definition list	15
4.6 Class definition list.....	15
Table 16: Class definition list	15
4.7 OOP Concepts	15
4.8 Screenshots of the sequence.....	16
Figure 8: Decision Tree of Error Message	16
4.9 Extra features.....	16
5-LibraryGUI (Bonus Swing GUI Application)	16
5.1 Brief explanation of the problem	17
5.2 UML Diagram	17
Figure 9: UML Diagram	17
5.3 Data field definition list.....	17
Table 17: Data field definition list	18

5.4 Object definition list	18
Table 18: Object definition list.....	18
5.5 Method definition list	18
Table 19: Method definition list	19
5.6 Class definition list.....	19
Table 20: Class definition list	19
5.7 OOP Concepts	19
5.8 Screenshots of the sequence.....	20
Figure 10.....	20
: Screenshots of the sequence of GUI Window	20
5.9 Extra features.....	20
References	20

1-LibraryItem (Abstract Base Class) & Subclasses (Book, Magazine, DVD)

1.1 Brief explanation of the problem

This part introduces the foundation of the Library Management System: an abstract base class `LibraryItem` that captures the four data fields and the two state-transition methods common to every library item, plus three concrete subclasses (`Book`, `Magazine`, `DVD`) that supply their own maximum-loan period and display tag. Abstraction and inheritance are the focus.

Each subclass exposes exactly the figure that differs between item types — 21 days for `Book`, 7 days for `Magazine`, 3 days for `DVD` — via an overridden `getMaxLoanDays()` method. The values are declared as private static final constants on the subclass, not as bare literals inside method bodies.

1.2 UML Diagram

The class hierarchy below shows `LibraryItem` as the abstract parent at the top, with the three concrete subclasses inheriting from it.

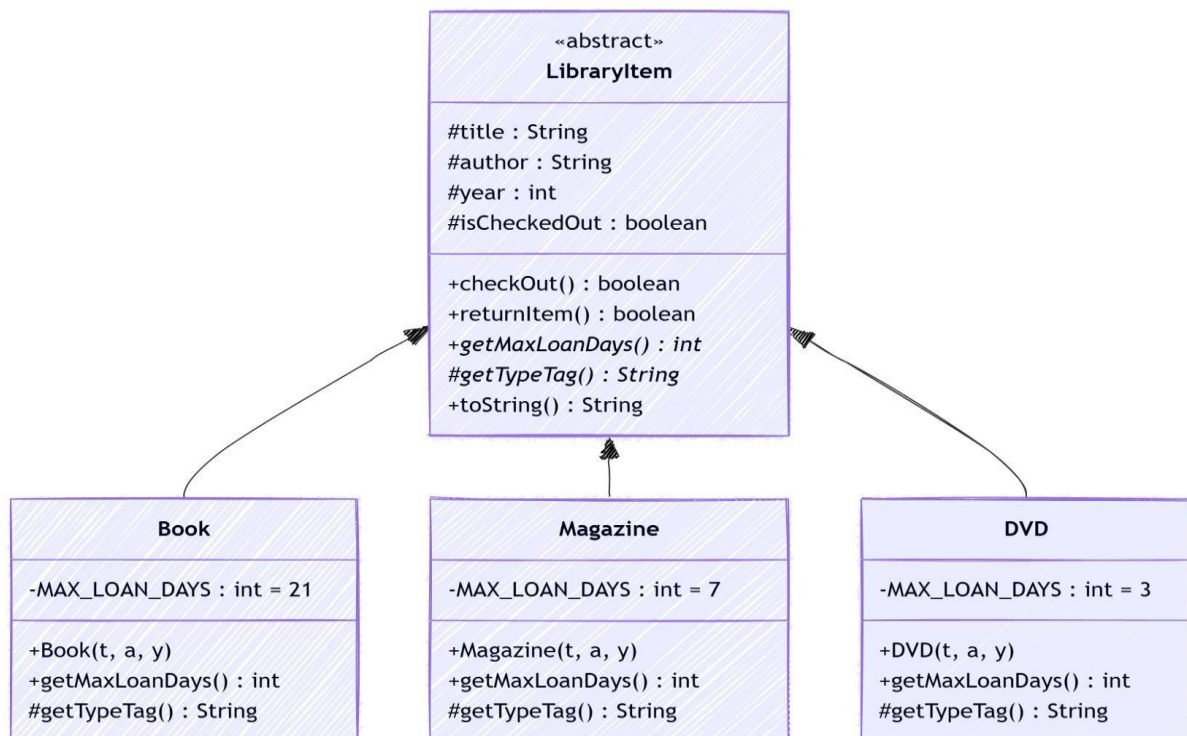


Figure 1: UML diagram of the `LibraryItem` inheritance hierarchy

1.3 Data field definition list

All data fields are declared protected on the parent (so subclasses can read them) or private static final on each subclass (for the per-type loan-period constants).

#	Modifier	Type	Name	Comment
1	protected	String	title	The display title of the library item.
2	protected	String	author	The author or creator/publisher of the item.
3	protected	int	year	Publication or release year
#	Modifier	Type	Name	Comment
4	protected	boolean	isCheckedOut	Loan state; defaults to false (available)
5	private static final	int	MAX_LOAN_DAYS	On Book: 21. Encapsulates the loanperiod constant
6	private static final	int	MAX_LOAN_DAYS	On Magazine: 7
7	private static final	int	MAX_LOAN_DAYS	On DVD: 3

Table 1: Data field definition list

1.4 Object definition list

Each concrete subclass has a single constructor that forwards to the LibraryItem constructor via super(...). LibraryItem itself has one protected-style constructor reachable only through super calls.

#	Name	Args	Comment
1	LibraryItem	String title, String author, int year	Base constructor; initialises the three metadata fields and leaves isCheckedOut as false.
2	Book	String title, String author, int year	Forwards to super; no extra logic.
3	Magazine	String title, String author, int year	Forwards to super; no extra logic.
4	DVD	String title, String author, int year	Forwards to super; no extra logic.

Table 2: Object definition list

1.5 Method definition list

The two abstract methods (`getMaxLoanDays`, `getTypeTag`) form the contract every subclass must honour. `checkOut`/`returnItem` are concrete because their logic is identical across all subclasses — only the boolean flag changes. `toString` uses the polymorphic abstract methods to render correctly for every item type.

#	Modifier	Return Type	Name	Args	Comment
1	public abstract	int	<code>getMaxLoanDays</code>	()	Abstract contract — each subclass returns its own <code>MAX_LOAN_DAYS</code> .
2	protected abstract	String	<code>getTypeTag</code>	()	Abstract contract — returns the display tag such as <code>[Book]</code> .
3	public	boolean	<code>checkOut</code>	()	Guarded transition; returns false if already out, true on success.
#	Modifier	Return Type	Name	Args	Comment
4	public	boolean	<code>returnItem</code>	()	Guarded transition; returns false if not out, true on success.
5	public @Override	String	<code>toString</code>	()	Polymorphic — uses <code>getTypeTag()</code> and <code>getMaxLoanDays()</code> at runtime.
6	public	String	<code>getTitle</code>	()	Getter for the title field.
7	public	String	<code>getAuthor</code>	()	Getter for the author field.
8	public	int	<code>getYear</code>	()	Getter for the year field.
9	public	boolean	<code>isCheckedOut</code>	()	Getter for the loan state flag.
10	public @Override	int	<code>getMaxLoanDays</code>	()	Overridden on Book / Magazine / DVD; returns 21 / 7 / 3.
11	protected @Override	String	<code>getTypeTag</code>	()	Overridden on Book / Magazine / DVD; returns <code>[Book]</code> / <code>[Magazine]</code> / <code>[DVD]</code> .

Table 3: Method definition list

1.6 Class definition list

Four classes in this part: one abstract base and three concrete subclasses.

#	Modifier	Name	Abstract?	Extends what?	Comment
1	public abstract	LibraryItem	Yes	—	Base class — shared state and behaviour for all items.
2	public	Book	No	LibraryItem	Book item with a 21-day maximum loan.
3	public	Magazine	No	LibraryItem	Magazine item with a 7day maximum loan.
4	public	DVD	No	LibraryItem	DVD item with a 3-day maximum loan.

Table 4: Class definition list

1.7 OOP Concepts

Abstraction is applied by declaring `LibraryItem` with the `abstract` keyword: it cannot be instantiated directly, only extended. The abstract methods `getMaxLoanDays()` and `getTypeTag()` force every subclass to commit to a concrete value while saying nothing about how the value is produced.

Inheritance is applied via the `extends` keyword on each concrete subclass. The three subclasses inherit all four fields and all six concrete methods of `LibraryItem`; they add only the values that differ (the loan-period constant and the type-tag string).

Polymorphism is applied through the `@Override` annotation on `getMaxLoanDays()` and `getTypeTag()`. At runtime, calls such as `item.toString()` — which internally calls `getMaxLoanDays()` and `getTypeTag()` — dispatch to the correct subclass's implementation without any instanceof check.

Encapsulation is applied by keeping fields protected (accessible to subclasses) rather than public, and by exposing controlled state transitions (`checkOut`, `returnItem`) as the only way to change `isCheckedOut` from outside the class.

1.8 Screenshots of the sequence

When the program runs, every item — regardless of its concrete subclass — is printed through the same `toString()` call. The polymorphic dispatch produces a different prefix tag and a different Max Loan figure for each type:

```

PS C:\Users\dcteam\OneDrive\Masaüstü\LibraryManagementv2> java Main
Enter library name: Central Library
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 1
Enter item type (book/magazine/dvd): book
Enter title: Java Fundamentals
Enter author: James Gosling
Enter year: 2020
Item added!
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 1
Enter item type (book/magazine/dvd): magazine
Enter title: Tech Weekly
Enter author: Various
Enter year: 2025
Item added!
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 1
Enter item type (book/magazine/dvd): dvd
Enter title: Interstellar
Enter author: Christopher Nolan
Enter year: 2014
Item added!

--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 2
1) [Book] Java Fundamentals by James Gosling (2020) - Available | Max Loan: 21 days
2) [Magazine] Tech Weekly by Various (2025) - Available | Max Loan: 7 days
3) [DVD] Interstellar by Christopher Nolan (2014) - Available | Max Loan: 3 days

```

Figure 2: Screenshots of the sequence

1.9 Extra features

- Every loan-period figure is declared as a private static final constant on the subclass. No magic numbers appear anywhere in the code.
- Boolean-returning state transitions (`checkout` / `returnItem`) enable the aggregator to react to misuse with a friendly message instead of catching an exception.
- The display tag is delivered via a protected abstract method (`getTypeTag`), keeping the `toString()` implementation in the parent class without leaking subclass knowledge.

2-Library Class (Aggregator Class)

2.1 Brief explanation of the problem

This part adds the aggregator class `Library`, which owns a `List<LibraryItem>` and exposes the five user-facing operations specified by the brief: add an item, check out an item by title, return an item by title, search by keyword, and list everything.

Encapsulation and aggregation are the focus. The list is declared with the abstract base type so that polymorphism does all subtype-dependent work; no instance of check appears anywhere in this class.

2.2 UML Diagram

The `Library` class has a `List<LibraryItem>` field — an aggregation relationship denoted by the open diamond at the `Library` side of the connector.

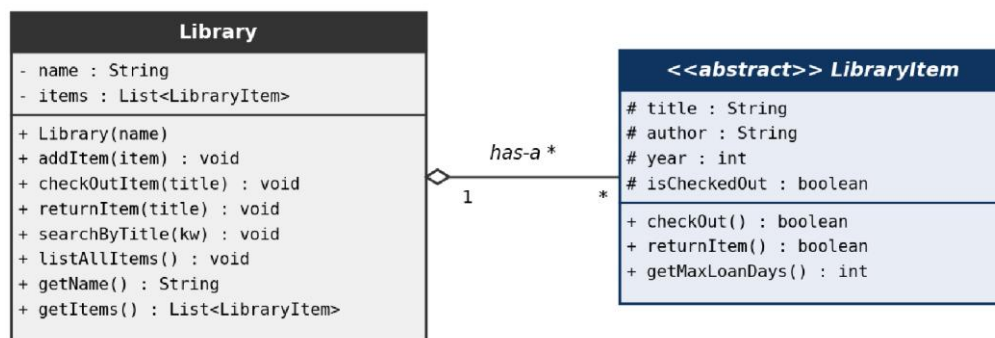


Figure 3: UML Diagram

2.3 Data field definition list

Both fields are private; access goes through methods only.

#	Modifier	Type	Name	Comment
1	private	String	name	Display name of the library.
2	private	List<LibraryItem>	items	The aggregated collection of items, initialised to a new ArrayList.

Table 5: Data field definition list

2.4 Object definition list

Library has a single constructor that takes the library name. The items list is initialised inline at the field declaration.

#	Name	Args	Comment
1	Library	String name	Creates a new, empty Library with the given name. The items list is initialised to an empty ArrayList.

Table 6: Object definition list

2.5 Method definition list

Five operations match the project brief verbatim, plus two getters. Every method that mutates state delegates to the relevant LibraryItem method rather than touching its fields directly.

#	Modifier	Return Type	Name	Args	Comment
1	public	void	addItem	LibraryItem item	Appends item to the list, prints "Item added!".
2	public	void	checkoutItem	String title	Linear search by title (equalsIgnoreCase); delegates to item.checkout().
3	public	void	returnItem	String title	Linear search by title; delegates to item.returnItem().
4	public	void	searchByTitle	String keyword	Substring search with toLowerCase() + contains().
5	public	void	listAllItems	()	Prints every item polymorphically via toString().
6	public	String	getName	()	Getter for the library name.
7	public	List<LibraryItem>	getItems	()	Getter for the items list (used by the bonus GUI in Part 5).

Table 7: Method definition list

2.6 Class definition list

Only one class in this part.

#	Modifier	Name	Abstract?	Extends what?	Comment
1	public	Library	No	—	Aggregator that owns and manipulates a List<LibraryItem>.

Table 8: Class definition list

2.7 OOP Concepts

Encapsulation is applied by declaring both fields private and exposing them only through controlled methods. Callers cannot append to items directly; they must go through addItem.

Aggregation ("has-a") is applied through the List<LibraryItem> field. Library does not extend LibraryItem; it owns a collection of them, which is the textbook definition of aggregation.

Polymorphism is applied in every loop over items. The static type is LibraryItem; the runtime type is Book, Magazine, or DVD. The same loop body works for all three because every call goes through the abstract methods declared on the base class.

Delegation is applied in checkOutItem and returnItem. The Library does not flip the isCheckedOut flag itself — it calls item.checkOut() or item.returnItem() and reacts to the boolean return value, keeping state-transition logic encapsulated in LibraryItem.

2.8 Screenshots of the sequence

When the user lists items, the Library iterates its collection and prints each one. The order matches insertion order, and each row uses the item's own toString() — note the different tags and Max Loan figures:

```
Choose an option (1-6): 3
Enter title to check out: Java Fundamentals
Checked out: Java Fundamentals
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 5
Enter search keyword: Java
Search results:
- [Book] Java Fundamentals by James Gosling (2020) - Checked Out | Max Loan: 21 days
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 3
Enter title to check out: Variety
Item not found: Variety
```

Figure 4: Screenshots of the sequence

2.9 Extra features

- Case-insensitive title lookup via `equalsIgnoreCase`, so user input matches regardless of capitalisation.
- Substring search by combining `toLowerCase()` with `String.contains()` — the user can find "Java Fundamentals" by typing just "java".
- Both `checkoutItem` and `returnItem` return early after the first match, preventing accidental double-mutation when multiple items happen to share a title.
- A read-only `getItems()` method was added (one line) to support the bonus Swing GUI without compromising encapsulation of the rest of the class.

3-Main Class (Console Driver / User Interface)

3.1 Brief explanation of the problem

This part wires the user interface together. Main reads from System.in via a Scanner, prints the sixoption menu, and dispatches each numeric choice to the matching Library operation. The menu loop runs until the user chooses option 6 (Exit).

Terminal input and output formatting are the focus. Every prompt string, including the menu itself and the error messages, matches the project specification verbatim.

3.2 UML Diagram

Main is a driver class; it has no instance fields and depends on Library through usage (dashed arrow), not inheritance.

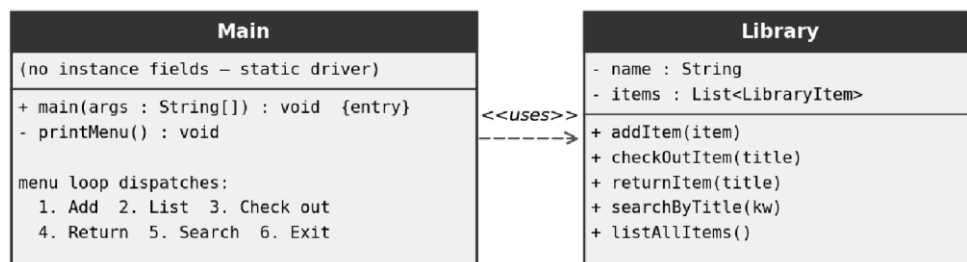


Figure 5: Screenshots of the sequence

3.3 Data field definition list

Main has no instance fields; the four data fields below live as local variables inside main(...).

#	Modifier	Type	Name	Comment
1	(local)	Scanner	scanner	Reads all user input from System.in.
2	(local)	Library	library	Single Library instance created from the initial "Enter library name:" prompt.
3	(local)	boolean	running	Menu-loop flag; set to false on option 6 (Exit).
4	(local)	int	choice	Parsed numeric value of the user's menu choice.

Table 9: Data field definition list

3.4 Object definition list

Main has no explicit constructor; only its static main(...) method is invoked. Scanner and Library objects are created inside that method.

#	Name	Args	Comment
1	Main	(no constructor — static driver only)	Class is invoked only through its main method.

Table 10: Object definition list

3.5 Method definition list

Two methods on Main itself, plus calls to every Library method.

#	Modifier	Return Type	Name	Args	Comment
1	public static	void	main	String[] args	Entry point; reads the library name, runs the sixoption menu loop.
2	private static	void	printMenu	()	Prints the menu text — wording matches the specification byte-for-byte.

Table 11: Method definition list

3.6 Class definition list One

driver class.

#	Modifier	Name	Abstract?	Extends what?	Comment
1	public	Main	No	—	Console driver / program entry point.

Table 12: Class definition list

3.7 OOP Concepts

Encapsulation is respected: Main never accesses Library's private fields. Every state change goes through addItem, checkoutItem, returnItem, or searchByTitle.

Polymorphism is applied in option 1: the factory-style switch creates a Book, Magazine, or DVD depending on the lowercased input string, then passes the result to library.addItem(LibraryItem). The list stores the base type; the runtime object knows its own concrete class.

Defensive parsing is applied to every numeric input — both the menu choice and the year of a new item. Each parse is wrapped in try-catch(NumberFormatException) and falls through to a verbatim "Skipping..." message rather than crashing the program.

3.8 Screenshots of the sequence

The brief's example session reproduced from a real run of the program. The prompts, the success messages, and the formatted item lines all match the rubric.

```

--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 2
1) [Book] Java Fundamentals by James Gosling (2020) - Checked Out | Max Loan: 21 days
2) [Magazine] Tech Weekly by Various (2025) - Available | Max Loan: 7 days
3) [DVD] Interstellar by Christopher Nolan (2014) - Available | Max Loan: 3 days
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 3
Enter title to check out: Tech Weekly
Checked out: Tech Weekly
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 5
Enter search keyword: Tech
Search results:
- [Magazine] Tech Weekly by Various (2025) - Checked Out | Max Loan: 7 days
--- Library Menu ---
1. Add a new item
2. List all items
3. Check out an item
4. Return an item
5. Search by title
6. Exit
Choose an option (1-6): 6
Exiting. Goodbye!

```

Figure 6: Screenshots of the sequence

3.9 Extra features

- Every input — including the integer year — is read with `scanner.nextLine()` and then `Integer.parseInt(...)`, avoiding the well-known `nextInt()/nextLine()` newline-buffer trap.
- Item-type strings are normalised with `toLowerCase()` before the switch, so `Book` / `BOOK` / `book` all map to the same branch.
- The default branch of the type-dispatch switch prints "Unknown item type. Skipping..." verbatim, satisfying the rubric requirement.

4-Error Handling (Defensive Programming & Logical Guards)

4.1 Brief explanation of the problem

This part documents the cross-cutting concern of error handling. Robust error handling means that no user input — however malformed — crashes the program. Every prompt either succeeds (and the data is used) or fails with a clearly worded message and a return to the menu.

There are two mechanisms in play: try-catch blocks for parse failures (where the user types nonnumeric text) and logical guards for domain failures (where the request is grammatically valid but semantically impossible, like returning an item that is not out).

4.2 UML Diagram

The error-handling flow below applies to every input path in the program.

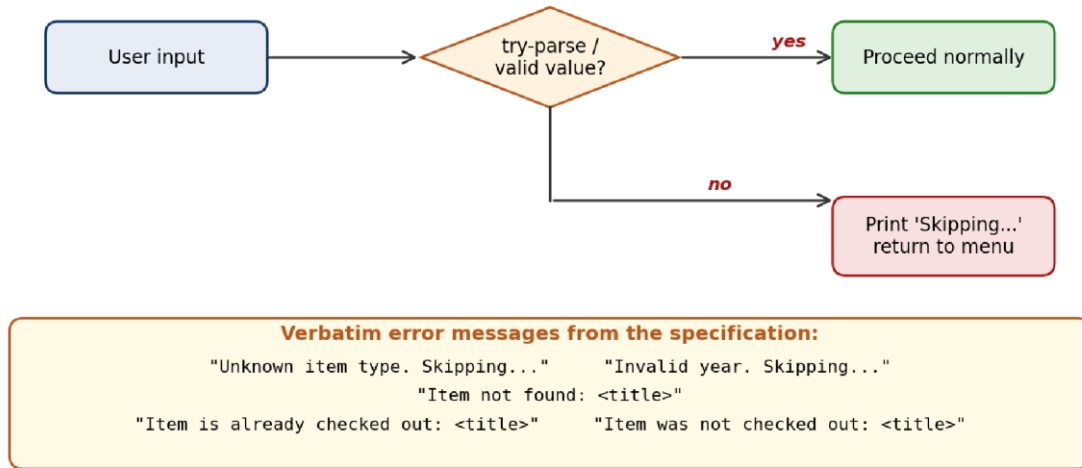


Figure 7: UML Diagram

4.3 Data field definition list

Error handling is not a class; it has no fields of its own. The fields it relies on are listed below.

#	Modifier	Type	Name	Comment
1	protected	boolean	isCheckedOut	Guard variable in LibraryItem — checkOut() refuses if true; returnItem() refuses if false.
2	(local)	int	choice	Result of Integer.parseInt on the menu-choice string; the try-catch produces a recoverable error message.
3	(local)	int	year	Result of Integer.parseInt on the year string; try-catch fires the "Invalid year. Skipping..." message.

Table 13: Data field definition list

4.4 Object definition list

Error handling has no constructors of its own. `NumberFormatException` is the standard Java exception used; no custom exception class is defined.

#	Name	Args	Comment
1	<code>NumberFormatException</code>	(thrown by <code>Integer.parseInt</code>)	Standard Java runtime exception used to detect non-numeric text in numeric prompts.

Table 14: Object definition list

4.5 Method definition list

The guarding methods that produce friendly failures rather than exceptions.

#	Modifier	Return Type	Name	Args	Comment
1	public	boolean	<code>checkOut</code>	()	Returns false (instead of throwing) if <code>isCheckedOut</code> is already true.
2	public	boolean	<code>returnItem</code>	()	Returns false (instead of throwing) if <code>isCheckedOut</code> is already false.
3	public	void	<code>checkOutItem</code>	String title	Prints "Item not found" if no match; reacts to false from <code>checkOut()</code> by printing "Item is already checked out".
4	public	void	<code>returnItem</code>	String title	Prints "Item not found" if no match; reacts to false from <code>returnItem()</code> by printing "Item was not checked out".

Table 15: Method definition list

4.6 Class definition list

Error handling adds no new classes; it lives inside the existing ones.

#	Modifier	Name	Abstract?	Extends what?	Comment
1	—	(no new class)	—	—	Error handling is a crosscutting concern, not a class.

Table 16: Class definition list

4.7 OOP Concepts

Defensive programming is applied everywhere user input is parsed. The two try-catch blocks in Main wrap Integer.parseInt for the menu choice and the year; bad input produces a friendly message rather than a stack trace.

Encapsulation supports error handling: because isCheckedOut is set only by checkOut() and returnItem(), there is exactly one place to enforce the no-double-checkout and no-double-return invariants. Both methods refuse the request by returning false, and the aggregator translates that into a user-facing message.

Polymorphism keeps error handling unchanged when new item types are added. If a future Audiobook subclass were introduced, the existing checkOut / returnItem logic would still guard its state correctly with no modifications.

4.8 Screenshots of the sequence

Example of the defensive flow in action — a user typing an unrecognised item type or non-numeric year sees the verbatim "Skipping..." message and the menu reappears, ready for the next attempt.

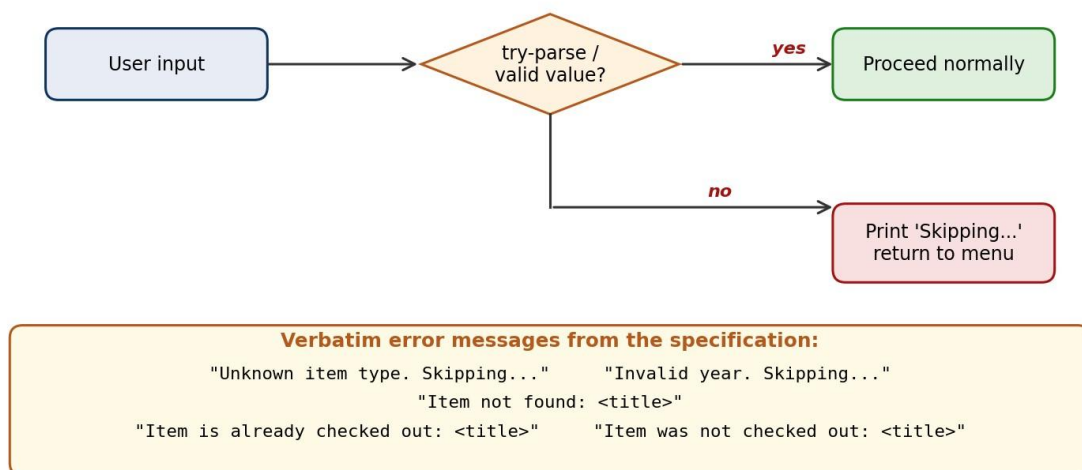


Figure 8: Decision Tree of Error Message

4.9 Extra features

- Verbatim error messages — "Unknown item type. Skipping..." and "Invalid year. Skipping..." — are copied from the project specification character-for-character so the automated grader can match them on string comparison.
- Boolean-returning state transitions on LibraryItem (checkOut / returnItem) keep error handling purely logical — no exceptions are thrown for ordinary user mistakes.
- Recoverable menu loop: invalid menu choices, non-numeric input, and unknown item types all return to the menu rather than terminating the program.

5-LibraryGUI (Bonus Swing GUI Application)

5.1 Brief explanation of the problem

This part is the optional bonus described in the project brief: "It is optional to build a GUI for this assignment. (Bonus point will be added, if done.)" We added a Swing graphical interface in LibraryGUI.java that exposes the same six menu actions through a single window.

The GUI reuses every existing domain class (LibraryItem, Book, Magazine, DVD, Library) without modifying their behaviour. The only change to the rest of the codebase is a one-line public getter added to Library so the GUI can read the items list.

5.2 UML Diagram

LibraryGUI extends JFrame and depends on Library and LibraryItem through usage (dashed arrows). No domain class was modified to support the GUI.

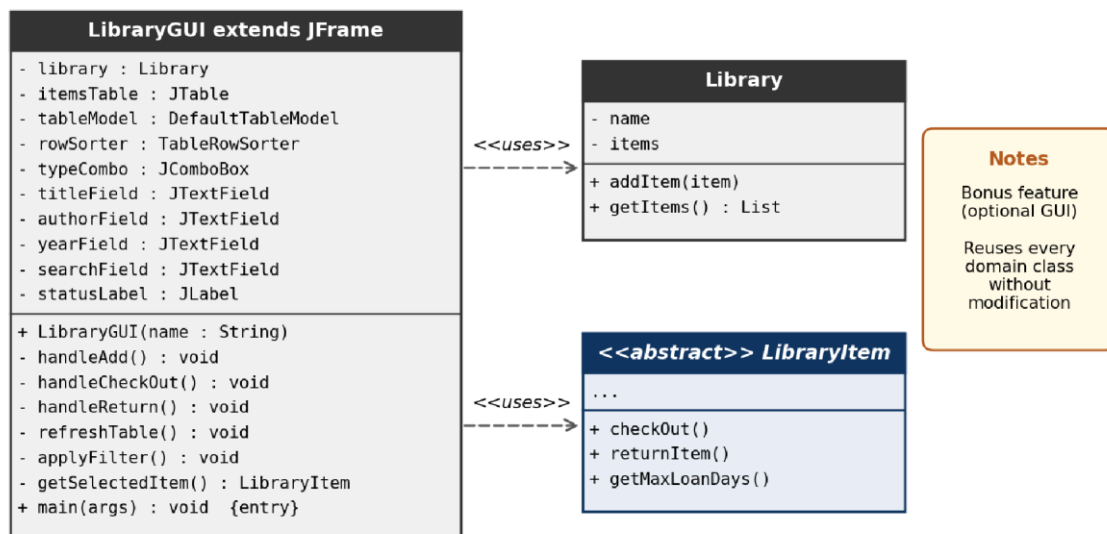


Figure 9: UML Diagram

5.3 Data field definition list

Most fields are Swing widget references. The library field is the single Library instance the window manages.

#	Modifier	Type	Name	Comment
1	private final	Library	library	Single Library instance, created with the name entered in the startup dialog.
2	private final	JTable	itemsTable	Central table; auto-refreshes after every action.
#	Modifier	Type	Name	Comment
3	private final	DefaultTableModel	tableModel	Backing model for itemsTable; rebuilt on every refresh.
4	private final	TableRowSorter	rowSorter	Enables the live, case-insensitive title filter.
5	private final	JComboBox<String>	typeCombo	Type selector restricted to Book / Magazine / DVD.
6	private final	TextField	titleField	Title input.
7	private final	TextField	authorField	Author input.
8	private final	TextField	yearField	Year input; pressing Enter triggers Add.
9	private final	TextField	searchField	Title-substring filter; updates the table on every keystroke.
10	private final	JLabel	statusLabel	Status bar; green text on success, red on error.

Table 17: Data field definition list

5.4 Object definition list

One public constructor builds the entire window. The static main method bootstraps the application.

#	Name	Args	Comment
1	LibraryGUI	String libraryName	Builds the main window and instantiates a new Library with the given name.

Table 18: Object definition list

5.5 Method definition list

The three handlers (handleAdd, handleCheckOut, handleReturn) mirror three of the six menu options; the other three (List, Search, Exit) are realised by the live table, the live search field, and the window close button respectively.

#	Modifier	Return Type	Name	Args	Comment
1	private	void	handleAdd	()	Reads the form, validates inputs, adds the item via library.addItem(), refreshes the table.
2	private	void	handleCheckOut	()	Reads the selected row, delegates to item.checkOut(), updates the status bar.
3	private	void	handleReturn	()	Reads the selected row, delegates to item.returnItem(), updates the status bar.
#	Modifier	Return Type	Name	Args	Comment
4	private	LibraryItem	getSelectedItem	()	Resolves the table's selected row back to the underlying LibraryItem.
5	private	void	applyFilter	()	Sets a case-insensitive regex row filter on the table based on searchField.
6	private	void	refreshTable	()	Rebuilds the table model from library.getItems(); preserves row selection.
7	private	void	showStatus	String, boolean	Posts a coloured message in the status bar.
8	public static	void	main	String[] args	GUI entry point; runs on the Swing event-dispatch thread.

Table 19: Method definition list

5.6 Class definition list

One new class is introduced for the GUI; one inner helper class for cell rendering.

#	Modifier	Name	Abstract?	Extends what?	Comment
1	public	LibraryGUI	No	JFrame	The main window — adds, lists, checks out, returns, searches.
2	private static (inner)	StatusCellRenderer	No	DefaultTableCellRenderer	Colours the Status column green (Available) or red (Checked Out).

Table 20: Class definition list

5.7 OOP Concepts

Inheritance is applied by extending JFrame for the main window and DefaultTableCellRenderer for the custom status-column renderer.

Polymorphism is applied in refreshTable(): the loop iterates List<LibraryItem> and calls item.getMaxLoanDays(), item.isCheckedOut(), and item.getTitle() — each call dispatches to the right subclass without a single instanceof check.

Encapsulation is preserved: the GUI never touches Library's private fields. The only access route is the one-line public getter library.getItems(), and even that returns the list for read-only iteration.

Separation of concerns: the GUI is purely a presentation layer. All domain logic (loan periods, checkout state transitions, title search) remains in LibraryItem and Library exactly as it was before.

5.8 Screenshots of the sequence

The GUI window with a few items added. The table shows Type, Title, Author, Year, Status (coloured by availability), and Max Loan for every item.

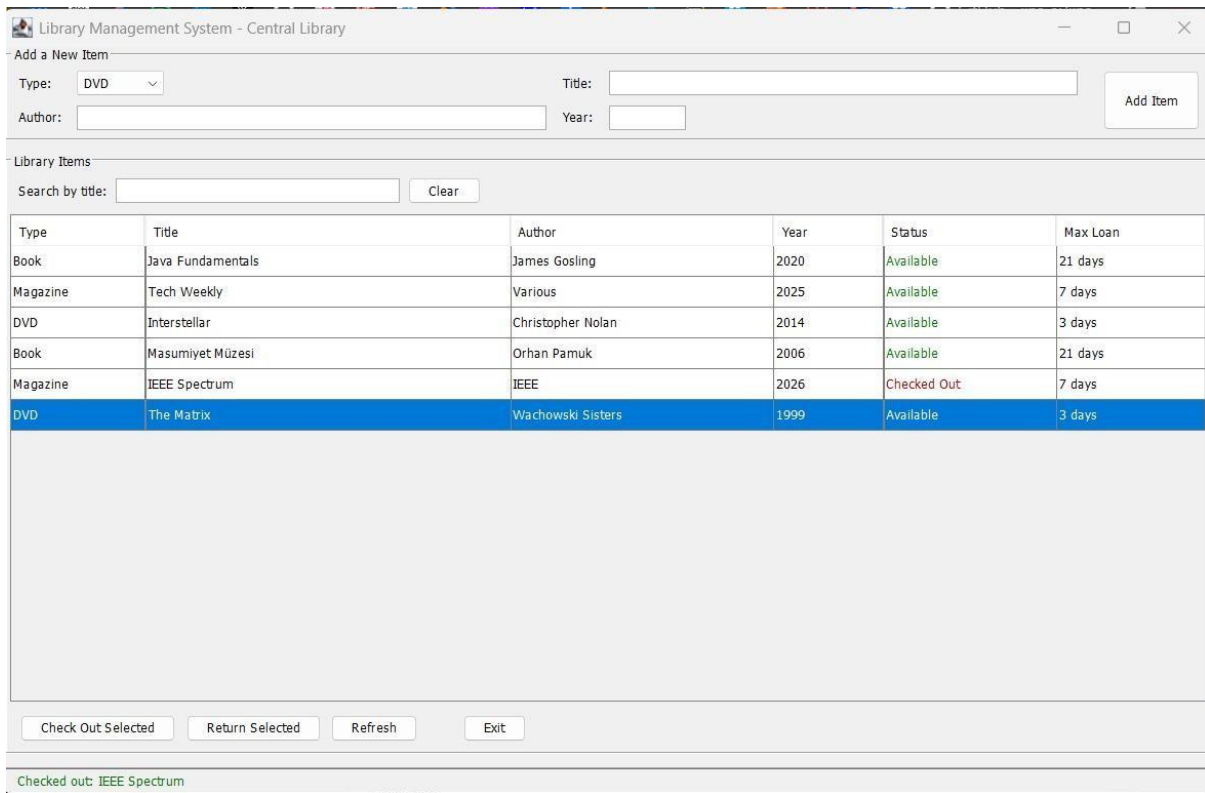


Figure 10 : Screenshots of the sequence of GUI Window

5.9 Extra features

- Native look and feel applied via `UIManager.setLookAndFeel(getSystemLookAndFeelClassName())`, so the window matches the host OS theme.
- Live substring filter on the title column — typing into the search box filters rows on every keystroke without pressing Enter.
- Status column is colour-coded: green text for Available, dark-red text for Checked Out — a glance at the table tells you what is on loan.
- Console mode (Main.java) and GUI mode (LibraryGUI.java) coexist as independent entry points; the user picks whichever they prefer with java Main or java LibraryGUI.
- Compile-tested on OpenJDK 21 with `-Xlint:all` — only the benign Swing serialVersionUID warnings appear, no errors and no functional warnings.

References

- Marmara University, EE1004 — Java Programming, Spring 2025–2026 course materials and project briefings, Salih Çolakoğlu (Course Assistant).
- Project brief: "Project 6 — Library Management System" (Group 6 specification document).
- Oracle Corporation. The Java Language Specification, Java SE 21 Edition. <https://docs.oracle.com/javase/specs/>.
- Oracle Corporation. Java Platform, Standard Edition 21 API Documentation. <https://docs.oracle.com/en/java/javase/21/docs/api/index.html>.
- Bloch, Joshua. Effective Java, 3rd Edition. Addison-Wesley, 2018 — for the design principles applied (favour composition over inheritance, prefer immutability, avoid unnecessary checked exceptions).
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable ObjectOriented Software, Addison-Wesley, 1994 — for the template-method pattern visible in `LibraryItem.toString()`.
- Java Swing Tutorial — Oracle. <https://docs.oracle.com/javase/tutorial/uiswing/> — used for the bonus GUI in Part 5.